Action Plan

Antenna Recommender (ANT)

Document ID	Phase	Version
ANT-EXEC-AP	Executing	1.0

1. Epic 1: System Architecture & Data Modeling

1.1 Story 1: System Architecture & Single Source of Truth

User Story: *As a **developer**, I want a well-defined system architecture and a single source of truth for antenna data, so that all system functions can reliably access and filter consistent, validated data.*

Tasks

- Define Technical Benchmarking & System Design Strategy (Activities 300, 310)
- Design Database Schema & Core Data Structures (antennaData JS Array Structure) (Activities 320, 350)
- Complete Technical Dataset Array (Dipole, Monopole, Yagi-Uda, Parabolic Dish, Patch Microstrip, Loop, Helical) with mapped parameters (Frequency, Gain, Polarization, Application, Coverage Pattern, Installation Type)
- Establish Single Source of Truth Architecture for data handling and filtering

Acceptance Criteria

- antennaData JSON structure fully populated with validated RF parameters
- Single Source of Truth data structure integrated and accessible by all system functions

2. Epic 2: Core Development & Algorithm Logic

2.1 Story 1: Recommendation Engine & Filtering Logic

User Story: *As an **end user**, I want the system to filter and recommend antennas based on my technical requirements (frequency, polarization, coverage, installation type), so that I get accurate antenna suggestions without manually researching specs.*

Tasks

- Design and implement Decision Logic / Algorithm (filterAntennas()) (Activity 330)
- Implement multi-parameter filtering logic for Frequency Ranges (LF, MF, HF, VHF, UHF, SHF)
- Develop dependent cascading selection logic linking Frequency Ranges to relevant Primary Applications
- Implement parameter matching for Polarization Types (Linear, Circular)
- Implement matching for Coverage Patterns (Omnidirectional, Directional) and Installation Types (Fixed Outdoor, Mobile, Embedded)
- Build dynamic fallback algorithm to present general-purpose antennas (e.g., Dipole/Monopole) when zero direct matches occur

Acceptance Criteria

- System correctly filters and recommends antennas matching all user-defined parameters with accuracy
- Fallback logic successfully triggers and displays valid alternative choices when no direct match is found

2.2 Story 2: Development Environment & Repository Setup

User Story: *As a **developer**, I want a secure, properly configured development environment and repository, so that the team can collaborate safely without exposing sensitive credentials or breaking the codebase.*

Tasks

- Setup Development Environment, Local Servers, and Workspace Security (Activity 340)
- Establish Git Branching Strategy (main, dev, feature/*)
- Configure Security Controls for API Key management

Acceptance Criteria

- GitHub repository configured with branch protection rules
- Development workspace running securely with API key environment safeguards

3. Epic 3: Frontend & Multi-language UI (i18n) Development

3.1 Story 1: Responsive UI & Component Architecture

User Story: *As an **end user**, I want a responsive, clearly presented interface with detailed result cards and comparison views, so that I can easily review and compare antenna options on any device.*

Tasks

- Create UI Wireframes and Layout Mockups (Activity 450)
- Develop Responsive UI Layout using Tailwind CSS (Activity 460)
- Build Input Validation Handlers for technical parameter bounds (Activity 410)
- Build dynamic UI Results Card Component (specs badges, diagrams, pros/cons)
- Implement Side-by-Side Technical Comparison Table

Acceptance Criteria

- UI is 100% responsive across desktop, tablet, and mobile breakpoints
- Cards and Comparison View dynamically populate technical parameters based on recommendation engine outputs

3.2 Story 2: Internationalization Framework (i18n)

User Story: *As an **Arabic- or English-speaking user**, I want to switch the interface language and have the layout adjust direction automatically, so that I can use the tool comfortably in my preferred language.*

Tasks

- Develop Bilingual Switcher Framework (toggleLanguage() / setLanguage()) supporting Arabic and English
- Implement dynamic Text Direction handling (RTL for Arabic, LTR for English) via DOM manipulation
- Build Localized Data Mapping Dictionary for strings, terminology, and error messages

Acceptance Criteria

- Language toggle switches language and text alignment (RTL/LTR) instantly without breaking UI layouts

4. Epic 4: Gemini AI Feature Integration

4.1 Story 1: AI Project Analyzer & Structured Output

User Story: *As an **end user**, I want to describe my project in plain language and have the AI extract my technical requirements automatically, so that I don't have to manually fill in every filter field myself.*

Tasks

- Integrate Gemini API Endpoint via JavaScript fetch (callGeminiStructured())
- Build Prompt Logic & Response Schema (analysisSchema) to parse unstructured descriptions into technical filter keys
- Implement analyzeProjectWithAI() to extract Frequency, Application, Polarization, Coverage, and Installation constraints
- Implement auto-fill UI handler to update input dropdowns based on Gemini's structured response

Acceptance Criteria

- AI Analyzer accepts natural language text, parses technical specs accurately, and auto-populates input controls

4.2 Story 2: Interactive AI Installation Guide Generator

User Story: *As an **end user**, I want to receive a clear, step-by-step installation guide for my recommended antenna, including safety notes and Arabic translation, so that I can install it correctly and safely without external help.*

Tasks

- Implement asynchronous API handler (callGemini()) to request custom installation guides
- Develop prompt engineering logic requesting structured Markdown, safety precautions, and Arabic translation
- Implement dynamic Markdown-to-HTML parser for headings, bullet points, and icons
- Build custom SVG loading spinner animation (getAntennaLoaderMarkup()) during API fetch states

Acceptance Criteria

- AI Installation Guide generates properly formatted step-by-step instructions with custom styling and zero rendering errors

5. Epic 5: Quality Assurance, Debugging & Code Optimization

5.1 Story 1: Unit Testing, Debugging & Refactoring

User Story: *As a **QA engineer / developer**, I want the system thoroughly tested, refactored, and documented, so that it runs reliably across browsers with no critical errors before delivery.*

Tasks

- Write and execute JavaScript Unit Tests (Activities 390, 400)
- Perform Code Freeze and Code Review (Activity 440)
- Execute Refactoring & Code Optimization (Activity 420)
- Add Inline Code Comments to maintain code quality (Activity 430)
- Conduct System Integration Testing (Activity 470) and User Acceptance Testing (UAT) (Activity 480)
- Verify latency requirements and cross-browser performance (Chrome, Firefox, Edge)

Acceptance Criteria

- Zero critical errors in Browser Console during cross-browser execution
- Recommendation response latency is under normal load
- Source code is fully refactored and includes code documentation comments